Universidade Federal de Campina Grande

SiGN – Sistema Gerenciador de Notas

Disciplina:

Introdução à Programação

Docente:

André Almeida

Discentes:

Caio Roberto da Silva Gomes

Gabriel Carvalho Rodrigues De Sousa

Campina Grande - PB

Março de 2026

Sumário

Sumário.....2
Introdução.....3
Objetivo do Sistema.....4
Como o Sistema Funciona.....4
Estrutura de Dados do Sistema.....5
- Notas.....5
- Frequência.....5

Função de Cálculo de Média.....6
- sum(notas).....6
- len(notas).....6
- Divisão para calcular a média.....7

Estrutura Principal do Programa.....7
Explicação de Cada Opção do Sistema.....7
- Visualizar alunos da turma.....7
- Ranking da Turma.....8
- Frequência dos Alunos.....9
- Alunos Aprovados.....10
- Alunos em Recuperação.....10
- Relatório Geral da Turma.....11
- Encerrar o Programa.....12
- Tratamento de Erros.....13

Como Executar o Sistema.....13
1. Instalar o Python.....13
2. Salvar o arquivo.....13
3. Executar no terminal.....14

Possíveis Melhorias Futuras.....14

Introdução

O **SiGN (Sistema Gerenciador de Notas)** é um pequeno sistema desenvolvido em **Python** que tem como objetivo auxiliar professores no acompanhamento básico de uma turma.

O sistema permite visualizar alunos, verificar médias, analisar frequência, identificar alunos aprovados ou em recuperação e gerar um relatório geral da turma.

Ele funciona **inteiramente pelo terminal (linha de comando)**, utilizando um **menu interativo** onde o usuário escolhe a ação desejada digitando um número.

Mesmo sendo um sistema simples, ele demonstra conceitos importantes da programação, como:

- Armazenamento de dados;
- Uso de funções;
- Estruturas de repetição;
- Tomada de decisão;
- Tratamento de erros.

Objetivo do Sistema

O objetivo principal do sistema é **organizar e apresentar informações acadêmicas de alunos de forma simples e rápida**.

Entre as principais funções do sistema estão:

- Visualizar todos os alunos da turma;
- Calcular a média das notas;
- Mostrar um ranking de desempenho;
- Verificar a frequência dos alunos;
- Identificar quem está aprovado;
- Identificar quem está em recuperação;
- Gerar estatísticas gerais da turma.

Como o Sistema Funciona

Quando o programa é executado, ele apresenta um menu de opções para o professor. Onde o professor pode escolher o que deseja fazer digitando um número correspondente à opção desejada.

Exemplo do menu exibido:

```
Olá, professor! Seja bem vindo ao 'SiGN'. O que você deseja fazer hoje?


1) - Visualizar alunos da turma
2) - Ranking da turma
3) - Frequência
4) - Aprovados
5) - Recuperação
6) - Gerar relatório
7) - Sair

Selecione a opção desejada:
```

Após escolher uma opção, o sistema executa a função correspondente.

O menu continua aparecendo até que o professor escolha a opção **7 (Sair)**.

Estrutura de Dados do Sistema

O sistema utiliza um **dicionário em Python** para armazenar as informações dos alunos.

Um dicionário é uma estrutura que guarda dados organizados em **pares de chave e valor**.

Nesse caso:

- A **chave** é o nome do aluno;
- O **valor** contém as notas e a frequência desse aluno.

Exemplo da estrutura utilizada:

```
alunos = {
  "Ana": {"notas": [8, 7, 9], "frequencia": 90},
  "Bruno": {"notas": [5, 6, 4], "frequencia": 75},
  "Carlos": {"notas": [9, 9, 10], "frequencia": 95},
  "Daniela": {"notas": [6, 5, 7], "frequencia": 80}
}
```

Cada aluno possui duas informações principais:

- **Notas**

  As notas são armazenadas em uma lista.

  Exemplo: [8, 7, 9] (Isso significa que o aluno possui três avaliações com essas notas).

- **Frequência**

  A frequência representa a porcentagem de presença do aluno nas aulas.

  Exemplo: 90 (Isso significa que o aluno esteve presente em 90% das aulas.)

Função de Cálculo de Média

O sistema possui uma função chamada media().

Funções são blocos de código criados para realizar uma tarefa específica.

Nesse caso, a função calcula a média das notas de um aluno.

Código da função:

```
def media(notas):
  return sum(notas) / len(notas)
```

Explicação passo a passo:

- **sum(notas)**

  A função sum() soma todos os valores da lista de notas.

  Exemplo:

  ```
  notas = [8, 7, 9]

  #8 + 7 + 9 = 24
  ```

  Resultado: 24

- **len(notas)**

  A função len() retorna a quantidade de elementos da lista.

  Exemplo:

```
notas = [8, 7, 9]

print(len(notas))
```

Resultado: 3

- **Divisão para calcular a média**

A média é calculada dividindo a soma das notas pela quantidade de notas.

Exemplo:

24 / 3 = 8

Logo, a média do aluno será:

8

Estrutura Principal do Programa

O sistema utiliza um loop infinito com o comando: “while True”. Isso significa que o programa continuará rodando continuamente até que o usuário escolha sair.

Esse tipo de estrutura é muito comum em programas que possuem menus interativos.

Explicação de Cada Opção do Sistema

- **Visualizar alunos da turma**

Essa opção mostra todos os alunos cadastrados no sistema.

O programa percorre o dicionário de alunos e imprime apenas os nomes.

Código:

```
for aluno in alunos:
    print(aluno)
```

Exemplo de saída:

```
Alunos da turma:


Ana
Bruno
Carlos
Daniela
```

- **Ranking da Turma**

Essa função organiza os alunos do melhor desempenho para o pior, baseado na média das notas.

O programa utiliza a função sorted(), que organiza dados automaticamente.

Código:

```
ranking = sorted(alunos items(),
                 key  lambda x  media(x[1]["notas"]),
                 reverse  True)
```

Exemplo de saída:

```
Ranking da turma:
```

```
1º - Carlos | Média: 9.33
2º - Ana | Média: 8.00
3º - Daniela | Média: 6.00
4º - Bruno | Média: 5.00
```

O ranking ajuda o professor a visualizar rapidamente quem possui melhor desempenho acadêmico.

- **Frequência dos Alunos**

Essa opção mostra a porcentagem de presença de cada aluno.

Código:

```
for nome, dados in alunos items():
        print(f"{nome}: {dados['frequencia']}%")
```

Exemplo de saída:

```
Frequência dos alunos:

Ana: 90%
Bruno: 75%
Carlos: 95%
Daniela: 80%
```

A frequência é importante porque muitas instituições exigem presença mínima para aprovação.

- **Alunos Aprovados**

Um aluno será considerado aprovado quando atender dois critérios simultaneamente:

- Média **maior ou igual a 7;**
- Frequência **maior ou igual a 75%.**

Código:

```
for nome, dados in alunos.items():

        if media(dados["notas"]) >= 7 and dados["frequencia"] >= 75:
            print(f"{nome} | Média: {media(dados['notas']):.2f}")
```

Exemplo de saída:

```
Alunos aprovados:


Ana | Média: 8.00
Carlos | Média: 9.33
```

- **Alunos em Recuperação**

Essa opção mostra alunos cuja média é menor que 7.

Código:

```
for nome  dados in alunos.items():

            if media  dados["notas"]      7
               print(f"{nome} | Média: {media(dados 'notas']):.2f}")
```

Exemplo:

```
Alunos em recuperação:


Bruno | Média: 5.00
Daniela | Média: 6.00
```

Isso permite ao professor identificar rapidamente quais alunos precisam de reforço ou recuperação.

- **Relatório Geral da Turma**

Essa opção gera estatísticas gerais da turma.

O sistema analisa todas as notas de todos os alunos.

Ele calcula:

- Média geral da turma;
- Maior nota registrada;
- Menor nota registrada.

Código:

```
todas_notas = []

        for dados in alunos.values():
            todas_notas.extend(dados["notas"])

        maior = max(todas_notas)
        menor = min(todas_notas)
        media_turma = sum(todas_notas) / len(todas_notas)
```

Exemplo de saída:

```
Relatório da turma:


Média geral da turma: 7.25
Maior nota da turma: 10
Menor nota da turma: 4
```

Esse tipo de relatório ajuda o professor a ter uma visão geral do desempenho da turma.

- **Encerrar o Programa**

Essa opção encerra o sistema.

Quando escolhida, o programa executa:

```
break
```

Esse comando interrompe o loop principal e finaliza o programa.

- **Tratamento de Erros**

O sistema utiliza um mecanismo chamado try / except.

Isso serve para evitar que o programa quebre caso o usuário digite algo inválido.

Exemplo:

Se o usuário digitar:

```
a, b, c, batata, etc
```

Em vez de um número, o programa exibirá a mensagem:

```
Erro! Digite apenas números.
```

Isso torna o sistema mais seguro e amigável para o usuário.

Como Executar o Sistema

Passo a passo:

1. **Instalar o Python**
    a. Primeiro é necessário ter o Python 3 instalado no computador.
    b. Site oficial:
    c. https://www.python.org
2. **Salvar o arquivo**
    a. Salve o código em um arquivo chamado, por exemplo:

    sign.py

3. **Executar no terminal**

    a. Abra o terminal ou prompt de comando e digite:

    python sign.py

    b. O menu do sistema aparecerá.

Possíveis Melhorias Futuras

Embora o sistema já funcione, ele pode ser expandido com várias melhorias, como:

- Cadastro de novos alunos;
- Edição de notas;
- Exclusão de alunos;
- Armazenamento em banco de dados;
- Interface gráfica;
- Geração de relatórios em PDF;
- Exportação para Excel;
- Gráficos de desempenho.

Essas melhorias tornariam o sistema mais próximo de um sistema acadêmico real.